

When to use different metrics

Metric	When to use
Accuracy	• Not well-suited to evaluation of IR tasks
Precision, Precision@k	• Unranked results, binary relevance • A fixed number of results is returned or of interest
F1-score	• Unranked results, binary relevance
R-relevance	• Unranked results, binary relevance • The number of relevant results in the corpus is known
MAP	• Ranked results, binary relevance • Interested in getting many relevant results on top • Weight more gains/errors on top rather than bottom of rank
MRR	• Ranked results, binary relevance • Interested in getting one relevant result on top
nDCG	• Ranked results, graded relevance • (Can be used for binary relevance too) • Weight more tail relevant documents than MAP

Combine multiple metrics for an all-round understanding of performance

Tolerant retrieval (fuzzy queries)

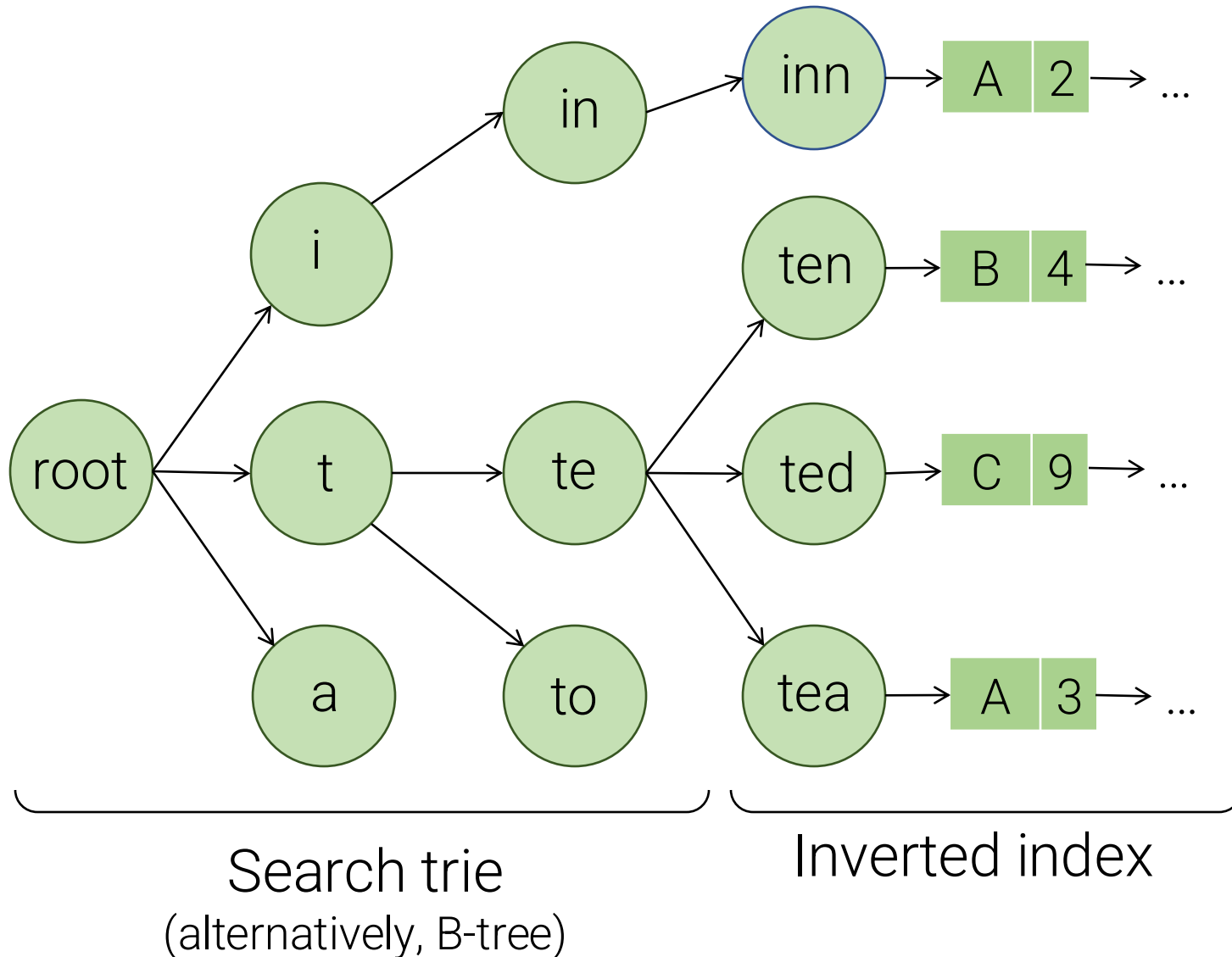
- Find the set of strings that closely match the query term
- Edit distance (Levenshtein distance):
 - The minimum number of single-character edits required to change one word into another

Replace k with m
A: k i t t e n s
B: m i t t e n
distance = 2

```
nltk.edit_distance(str1, str2)
```

- How do we implement it on an inverted index?

Tolerant retrieval (fuzzy queries)



- Input:
 - Query, max edit distance **d**
- Output:
 - Inverted index postings corresponding to terms within distance **d** from query
- Levenshtein automaton to prune irrelevant branches of the trie

References on fuzzy queries implementation

Michael McCandless (key contributor to Apache Lucene) on fuzzy search
<https://blog.mikemccandless.com/2011/03/lucenes-fuzzyquery-is-100-times-faster.html>

Implementation (roughly) explained
<https://julesjacobs.com/2015/06/17/disqus-levenshtein-simple-and-fast.html>

Theoretical paper (advanced)
<http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.16.652>

Elasticsearch Python API wrapper

```
# Low-level API wrapper - for higher level functions: pip install elasticsearch
from elasticsearch import Elasticsearch

# path to a certificate that will allow your API code to connect to your ES instance
path_to_ca_certificates = '/path/to/elasticsearch-8.5.3/config/certs/http_ca.crt'

# connect to the local elasticsearch node and authenticate with default user
# check documentation
es = Elasticsearch(hosts=['http://localhost:9200'], ca_certs=certificates,
                    http_auth=('elastic', 'YOUR_PWD_HERE'))

# es = Elasticsearch([{'host': 'localhost', 'port': 9200, 'scheme': 'http'}],
                    http_auth=(USER, PWD)) # alternative

# create the index - settings contains parameters (can be None)
es.indices.create(index=index_name, settings=settings)

# json-like objects
documents = [{'title' : 'henlo doggo'}, {'title' : 'are u fren?'}]
for i, doc in enumerate(documents):
    # index the documents with corresponding ids
    res = es.index(index=index_name, id=i, document=doc)
```

Elasticsearch with Requests

- Easier approach, more directly aligned with the API
- I prepared some convenience methods and object for you

```
# send a PUT request to create an index
r = requests.put('http://localhost:9200/my_new_index', # API endpoint
                json= None, # payload (in this case, empty)
                auth=HTTPBasicAuth(USER, PWD), # auth with password
                verify = self.path_to_ca_certificates) # certificate always needed
r.json()
```

```
# Elastic is a custom class I created for you
```

```
# send a PUT request to create an index
e = Elastic(ES_ENDPOINT, USER, PWD, path_to_ca_certificates)
# send a PUT request to create an index
r = es.curl('put', 'books')
# insert into index a document with id 123 whose content is in the variable doc
r = es.curl('put', 'books/_doc/123', json=doc)
r.json()
```

Search queries

```
# Full text query
GET books/_search {
  "query": {
    "match": {
      "title": {
        "query": "love magic"
      }
    }
  }
}
```

```
# exact search query
GET books/_search {
  "query": {
    "match_phrase": {
      "title": {
        "query": "love magic"
      }
    }
  }
}
```

```
# Full text query, multiple fields
with boosting
GET books/_search {
  "query": {
    "multi_match": {
      "query": "shakespeare",
      "fields":
        ["title",
         "author_name^1.5"],
      # add here "type": "phrase" for exact
      matching
    }
  }
}
```

```
#range query
GET books/_search
{
  "range": {
    "year": {
      "gte": 1000,
      "lte": 2000
    }
  }
}
```

Fuzzy queries

```
#range query
GET books/_search
{
  "query": {
    "fuzzy": {
      "title": {
        "value": "comander", #the query
        "fuzziness": "AUTO", #maximum edit distance allowed, AUTO
estimates based on length of term
        "max_expansions": 10, #maximum number of term variations created
        "prefix_length": 0, #characters left unchanged at start of term
        "transpositions": True, #to include transpositions in edits
      }
    }
  }
}
```


Aggregations

send a GET request for an aggregation query

```
GET books/_search {
  "aggregations": {
    "by_category": {
      "terms": {
        "field": "country",
        "size": 100
      }
    }
  }
}
```

```
'buckets': [
  {'key': 'England', 'doc_count': 41249},
  {'key': 'United States of America',
   'doc_count': 4558},
  {'key': 'Scotland', 'doc_count': 2357},
  {'key': 'England ; Scotland', 'doc_count':
   804},
  {'key': 'Ireland', 'doc_count': 678},
  {'key': 'England ; United States of
   America', 'doc_count': 566},
  {'key': 'Italy', 'doc_count': 495},
  {'key': 'Russia', 'doc_count': 324},
```

Get information from the Inverted Index

```
# Full text query
GET books/_termvectors/DOC_ID {
  "fields" : ["title"],
  "term_statistics" : True,
  "field_statistics" : True,
  "positions": True,
  "filter": {
    "min_word_length": 2,
    "min_term_freq" : 1
  }
}
```

```
{ '_index': 'books',
  ...
  'term_vectors': {
    'title': {
      'field_statistics': {
        'sum_doc_freq': 145690,
        'doc_count': 13077,
        'sum_ttf': 163599
      },
      'terms': {
        'death': {
          'doc_freq': 76,
          'ttf': 83,
          'term_freq': 1,
          'tokens': [{'position': 1,
            'start_offset': 4, 'end_offset': 9}],
          'score': 6.1348815}, ...
```

Exercise (in Python)

- Run jupyter notebook, solve exercises from A to G

id	author_name	title	country	language	year
14602826	Yearsley, Ann	Poems on several occasions	England	English	1786
14602830	A, T.	A Satyr against Vertue. (A poem: supposed to b...	England	English	1679
14602831	A, T.	The Aeronaut, a poem; founded almost entirely,...	Ireland	English	1816
14602832	Albert, Prince Consort, consort of Victoria, Q...	The Prince Albert, a poem	Ireland	English	1868

- Assignment
 - Index the whole dataset using given IDs
 - Make title and author full-text searchable and the country and language exact-text searchable
 - Find all poems published between 1850 and 1890 in England